INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD

Department of Computer Science and Engineering · Cloud Computing

CASE STUDY ASSIGNMENT — FINAL VERSION

Stay at the Helm

Secure Kubernetes Deployments via Graph Generation and Attack Reconstruction

A study of Blaise and Rebecchi (IEEE CLOUD 2022), and its extension in KubeAttackViz: from predefined static coefficients to learned, evidence-based edge weights, checked against attack paths documented by public benchmarks

Submitted by (Group 39)

Name	Roll No.
Ganesh Daware	23BCS038
Gagan Rathod	23BCS106
Venkatesh Dhangar	23BCS041
Advait Pathak	23BCS094

Submitted to

Dr. Malay Kumar

Department of Computer Science and Engineering, IIIT Dharwad

November 2026

Contents

Abstract 3

1. Introduction 3

1.1 Contributions of this final version . . . 3

2. Background 4

2.1 The problem with checklists . . . 4
2.2 The Kubernetes threat matrix . . . 4
2.3 Kubernetes security controls relevant to attack paths . . . 4

3. The reference paper: Stay at the Helm 5

3.1 Overview and contribution . . . 5
3.2 Threat model . . . 5
3.3 The method . . . 5
3.4 An illustrative example . . . 6
3.5 Evaluation . . . 7
3.6 The authors' conclusions . . . 8
3.7 Critical assessment . . . 8

4. Related work 9

5. Our implementation: KubeAttackViz with static coefficients 9

5.1 Mapping to the paper's four steps . . . 10
5.2 The six features in a live cluster . . . 10
5.3 The first version: predefined static coefficients . . . 10

6. Extension: learned, evidence-based edge weights 11

6.1 Design principle . . . 11
6.2 Why vulnerability counts are a poor proxy . . . 12
6.3 The CVE exploitation model . . . 12

7. Evaluation: static coefficients versus learned weights 13

7.1 Experimental setup . . . 13
7.2 Validation of the edges against a live cluster . . . 13
7.3 Public benchmark: Kubernetes Goat and Bad Pods . . . 14
7.4 Before and after on three live clusters . . . 17
7.5 Sensitivity analysis . . . 19
7.6 Real-time detection of a misconfiguration . . . 20

8. Demonstration 20

9. Discussion 21

9.1 Relationship to the paper . . . 21
9.2 Threats to validity . . . 22

10. Conclusion and future work 22

References 23

Abstract

This report studies “Stay at the Helm: secure Kubernetes deployments via graph generation and attack reconstruction” by Agathe Blaise and Filippo Rebecchi [1]. The paper proposes a four-step methodology for analysing Kubernetes Helm Charts: reconstruct a deployment graph, enrich it with six security analyses, assign node and edge weights with respect to the Kubernetes threat-matrix tactics, and reconstruct high-risk attack paths using Dijkstra’s algorithm after a log transformation. The evaluation covers 592 official Helm Charts from 14 public repositories, comprising 1,076 pods and 1,318 containers, and shows that connectivity, especially missing network policies and pod affinities, expands the set of containers reachable from an entry point by a factor of 2.35.

This is the final version of our case study. Like the paper, the first version of our implementation, KubeAttackViz [2], used predefined static coefficients. We keep the original case study and add an extension: every edge weight is derived from the probability that the attack step succeeds. Configuration-decidable steps are proven from live RBAC, NetworkPolicy and Pod Security state, and vulnerability steps use an exploitation model we trained on 361,825 NVD vulnerabilities labelled with the CISA Known Exploited Vulnerabilities catalogue. On a temporally held-out test set, the learned score raises ROC-AUC from 0.770 (CVSS score alone) to 0.887 and precision in the top 100 from 5% to 54%.

We then test both weightings against an answer key we did not write: the attack paths documented by two public, deliberately vulnerable projects, Kubernetes Goat and Bad Pods, deployed unmodified on a live cluster. Both weightings find all 7 documented Kubernetes Goat attacks, but the learned weights rank them at a mean position of 2.5 instead of 17.5 among 80 paths, and agree with 14 of 15 documented answers instead of 12. We also compare the weightings on three live clusters, validate the feasibility engine against 22,425 checks on a real cluster, report the sensitivity analysis the original paper did not include, and show that a misconfiguration introduced at runtime is detected within one monitoring interval and ranked first only by the learned weights.

Keywords: Kubernetes security, Helm, attack graphs, threat matrix, RBAC, Dijkstra, vulnerability exploitation prediction, machine learning, CISA KEV, Kubernetes Goat, Bad Pods.

1. Introduction

Kubernetes provides security mechanisms such as network segmentation, process isolation and role-based access control (RBAC), but the paper notes that these controls are effective only when configured correctly. Helm adds another layer of complexity because Charts package deployment resources into YAML-based configuration and can become very large; the authors report Charts of more than 40,000 lines [1]. The central question of the paper is therefore not simply whether an individual Kubernetes setting passes a security check, but how configuration properties interact and form multi-step attack paths.

The paper answers this question with a weighted graph. The weights, however, are predefined coefficients chosen by the authors (Table 2): for example, every high-severity vulnerability adds 1 to a container’s score and a missing network policy gives an edge probability of 0.8. Our first implementation followed the same design. When we presented it, the obvious follow-up questions were whether these numbers could be learned from real data instead of chosen by hand, and how we would know that the result is better and not only different. This final version answers both.

1.1 Contributions of this final version

- **Case study (retained from the first submission).** A structured explanation of the paper’s threat model, method, scoring model, evaluation and limitations (Section 3).

- **Static-coefficient baseline.** Our first KubeAttackViz version and how its fixed coefficients relate to the paper's (Section 5).
- **Learned edge weights.** A probabilistic edge model, w(e) = −ln P(e), identical in form to the paper's log(1/z(e)) transformation, where P(e) is either decided exactly from cluster configuration or estimated by a model trained on public exploitation data (Section 6).
- **External benchmark.** Both weightings checked against attack paths documented by Kubernetes Goat and Bad Pods, two public projects whose manifests we deploy unmodified (Section 7.3).
- **Live comparison and validation.** Before/after comparison on three live clusters, validation of the feasibility engine against a real cluster, a sensitivity analysis of the one remaining assumed parameter, and a runtime drift experiment (Sections 7.2 and 7.4–7.6).
- **Demonstration tool.** A single-command presentation of all of the above on live clusters (Section 8).

Section 2 introduces the background. Section 3 is the case study of the paper. Section 4 reviews related work. Section 5 presents KubeAttackViz with static coefficients, Section 6 the machine-learning extension and Section 7 compares the two. Section 8 describes the demonstration, Section 9 revisits the paper's limitations and Section 10 concludes.

2. Background

2.1 The problem with checklists

The paper groups Kubernetes configuration-auditing approaches into CIS-benchmark checks, custom best-practice checks and penetration-testing approaches. Examples discussed by the authors include kube-bench, kube-hunter, Checkov, kubeaudit and Polaris [1]. The limitation motivating the paper is that individual checks do not correlate findings into a single model of how weaknesses combine across components, and that results are not expressed in terms of attacker tactics and multi-stage paths.

2.2 The Kubernetes threat matrix

MITRE ATT&CK organises adversary behaviour into tactics and techniques [4]. Microsoft's threat matrix for Kubernetes adapts this structure to container orchestration [3]. It provides the vocabulary for stages such as initial access, execution, privilege escalation, credential access, discovery, lateral movement and impact. The paper does not use the matrix only as background: it incorporates tactic-specific scores into the graph model.

2.3 Kubernetes security controls relevant to attack paths

Three Kubernetes mechanisms determine most of the edges in an attack graph, both in the paper and in our implementation:

- **RBAC.** Roles and ClusterRoles list allowed verbs on resources; RoleBindings and ClusterRoleBindings grant them to users, groups and ServiceAccounts. A pod inherits the permissions of its ServiceAccount through the mounted token [6].
- **NetworkPolicy.** Without any policy selecting a pod, all ingress and egress traffic is allowed. This default is the reason the paper treats a missing network policy as a high-probability edge.
- **Pod security.** Root execution, privileged mode, Linux capabilities and host namespaces control whether a compromised container can reach its node. The paper analyses these via Pod Security Policies (PSP); PSP was removed in Kubernetes v1.25 and replaced by Pod Security Standards (PSS) and the Pod Security Admission controller [7].

3. The reference paper: Stay at the Helm

3.1 Overview and contribution

Blaise and Rebecchi, from Thales, presented the paper at the 2022 IEEE 15th International Conference on Cloud Computing (CLOUD), pages 59–69 [1]. The contribution is a methodology that converts a Helm Chart deployment into an enriched graph, evaluates its security properties against Kubernetes threat-matrix tactics, and reconstructs risky attack paths. The authors validate the approach on a dataset of public Helm Charts rather than only on a small hand-built example.

3.2 Threat model

The attacker is assumed to obtain an initial foothold in a container, typically through a service exposed outside the cluster (NodePort, LoadBalancer or Ingress), and then to move laterally between pods or towards the host. Internal services are not directly reachable, but the paper introduces a parameter ω: a non-zero probability that an attacker can reach a pod even if its service is internal, which models entry mechanisms not visible from service exposure alone (for example a compromised dependency).

3.3 The method

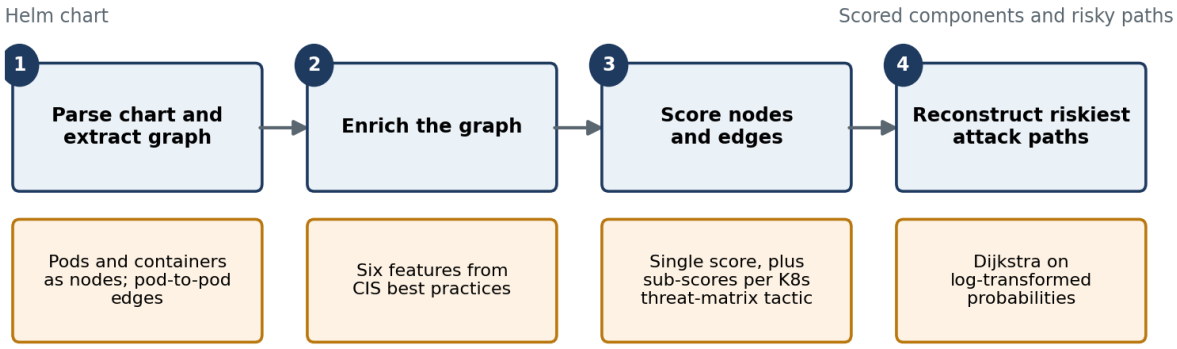


Figure 1. *The four-step methodology, redrawn from Figure 1 and Section IV of [1].*

Step 1: graph extraction. The Chart is parsed into a model containing 16 resource categories, including pods, containers, namespaces, services, service accounts, roles and bindings, pod security policies, network policies, secrets and volumes. The final topological graph has pods and containers as vertices. Each pod is connected to the containers it encapsulates, while pod-to-pod edges represent possible movement based on network policies, permissions to execute in other pods and pod co-location. Other Kubernetes resources are correlated to the pod they affect [1].

Step 2: enrichment with six security analyses. The graph is enriched using six feature groups inspired by the Kubernetes CIS benchmark [5] (Table 1). For vulnerabilities, the paper records high- or critical-severity findings (CVSS > 7) using scanners such as Trivy, Docker Scan and Snyk.

Table 1. The six enrichment features, after Section IV-B of [1].

#	Feature	What it captures
1	Container vulnerabilities	Count of high or critical CVEs per image, from open-source scanners
2	Pod accessibility	Whether services expose a pod (ClusterIP, NodePort, LoadBalancer, Ingress)
3	Security policies and contexts	Root or privileged execution, Linux capabilities, host network, PID and IPC, volume types
4	Access control (RBAC)	Service-account permissions such as reading secrets or creating pods
5	Firewall rules	Network policies governing which pods may talk to which
6	Pod affinities	Whether pods must, or must not, share a machine

Step 3: scoring. The paper defines two scoring systems. The single-weight system assigns one node weight and one edge weight; edge weights are probabilities in [0, 1] and multiply along a path. The multiple-weight system assigns each node a sub-score per threat-matrix tactic, while edges keep probability-style weights. The authors use nine tactics: Initial Access, Execution, Privilege Escalation, Defence Evasion, Credential Access, Discovery, Impact, Lateral Movement to Host and Lateral Movement to Pod. A node's score is a weighted sum of its findings, and an edge's probability combines the γ coefficients of the conditions that create it:

$$s(v) = \alpha_1 n_{vuln} + \alpha_2 n_{cap} + \sum_k \beta_k \, \mathbb{1}[\text{finding}_k(v)] \tag{1}$$

$$z(e) \in \{\gamma_1 = 0.8,\ \gamma_2 = 0.4,\ \gamma_3 = 1\}, \qquad z(\pi) = \prod_{e \in \pi} z(e) \tag{2}$$

The coefficients are configurable; the paper gives one concrete setting for its evaluation (Table 2).

Table 2. Predefined coefficients chosen by the authors (Table VI of [1], condensed).

Symbol	Meaning	Value
α_1 / α_2	Vulnerabilities / extra Linux capabilities	1 per vuln. / 5 per capability
β_1, β_2	Accessible service; ability to run as root	100 each
β_3, β_5	Can escalate privilege; can exec into other pods	100 each
β_4	Permissions (evasion, credential access, discovery, impact)	10 per tactic
β_6, β_7	Can reach host network; can reach host paths	100 each
β_8	Allowed host volume types	10 per volume
γ_1, γ_2, γ_3	No network policy; pod affinity; pod execution capacity	0.8; 0.4; 1

Step 4: attack-path reconstruction. The objective is to find acyclic paths with the largest product of edge probabilities. The paper turns this maximisation into a shortest-path problem by taking log(1/z(e)) as the edge cost; since $0 < z \leq 1$ every cost is non-negative and Dijkstra's algorithm applies:

$$\pi^* = \arg\max_{\pi} \prod_{e \in \pi} z(e) = \arg\min_{\pi} \sum_{e \in \pi} \log \frac{1}{z(e)} \tag{3}$$

This transformation is the bridge to our extension: Section 6 keeps exactly this form and changes only where z(e) comes from.

3.4 An illustrative example

The following example was constructed for this report to make the probability model intuitive; it is not an experiment reported by the authors. Consider a web pod exposed by a LoadBalancer, an internal API pod,

and a database pod whose service account can read secrets. If no network policy separates the pods, each edge has probability $\gamma_1 = 0.8$. The two-edge path has probability $0.8 \times 0.8 = 0.64$ and transformed cost $\log(1/0.8) + \log(1/0.8) \approx 0.446$. Individually moderate weaknesses combine into a multi-step route.

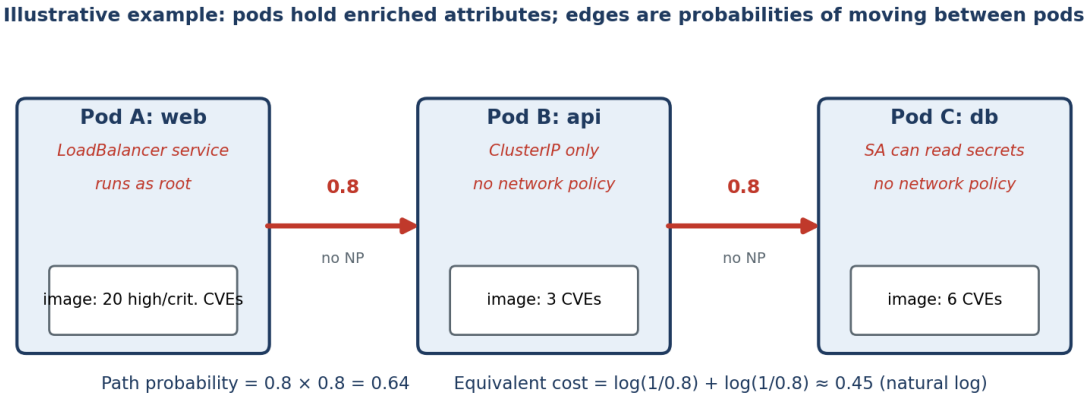


Figure 2. Illustrative example constructed for this report, using the paper's edge probability for a missing network policy.

3.5 Evaluation

Dataset. The evaluation scrapes Helm Charts from 14 open-source repositories: 592 official Charts, 1,076 pods and 1,318 containers. The largest Chart contains 43,628 lines; the average is 564 lines [1].

Compliance with the CIS benchmark. Table IV of the paper reports that 32.2% of containers run as root, 4.2% are privileged, 86.3% retain the default Linux capabilities and 88.8% allow NET_RAW; 9.8% may list secrets, 13.9% list pods, 8.1% create pods, 1.0% mount host paths and 0.8% use the host network. The median image has roughly 100 high- or critical-severity vulnerabilities, with examples above 3,000, and some images are opaque to the scanners [1].

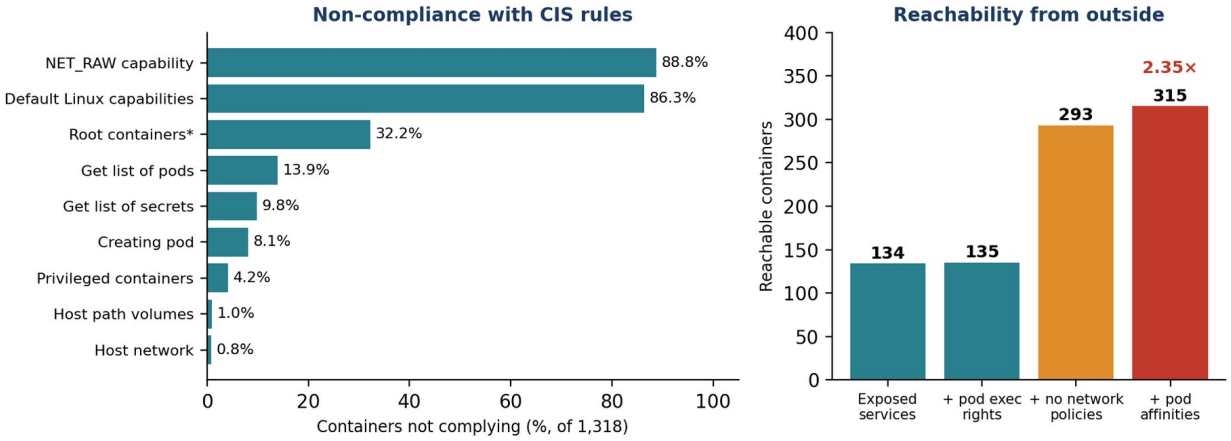


Figure 3. CIS non-compliance and reachability from outside, prepared for this report from Tables IV and V of [1].

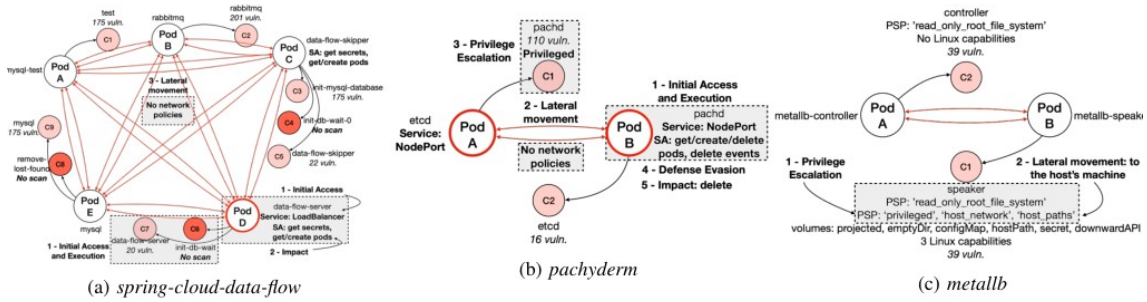
Connectivity is the key reason for the graph model. Starting from 134 containers reachable through exposed services, adding pod-execution permissions raises the count to 135, adding missing network policies to 293, and adding pod affinities to 315, a 2.35× increase over the initially reachable set [1].

Table 3. Containers vulnerable to each tactic under three settings (Table VII of [1], condensed).

Tactic	ω = 0	ω = 0, with paths	ω > 0
Initial access	122	122	1,318
Execution	103	252	1,112
Privilege escalation (capabilities)	4	13	92
Privilege escalation (run as root)	112	285	1,194
Permissions	23	43	224
Access to host network / paths / volumes	0 / 0 / 0	0 / 0 / 0	72 / 13 / 10

Case studies. Similar global scores can arise from different causes. The node-exporter pod in the Bitnami kube-prometheus Chart scores 470 and combines host-network, host-PID and host-volume access; the pachd pod in the Pachyderm Chart scores 411 and combines privileged execution with secret and pod permissions. In spring-cloud-data-flow, a path starts at a LoadBalancer, reaches an opaque container and a container with 20 high or critical vulnerabilities, then uses service-account permissions to read secrets and create pods. In pachyderm, a NodePort entry leads through missing network policies to a privileged pod; in metallb, security policies permit host-network access and host-volume mounting (Figure 4).

TABLE VII: Number of containers vulnerable to each K8s threat matrix tactic, for three configurations.



(a) *spring-cloud-data-flow* (b) *pachyderm* (c) *metallb*

Figure 4. *Attack-path examples from the paper's evaluation: spring-cloud-data-flow, pachyderm and metallb (Fig. 6 of [1]).*

3.6 The authors' conclusions

The authors conclude that the methodology can analyse hundreds of Helm Charts and expose security-sensitive configurations and attack paths. They identify three directions for future work: improving vulnerability analysis so that fewer images remain opaque, automated remediation and prioritisation of fixes based on reconstructed paths, and integration into a Kubernetes CI/CD security pipeline [1].

3.7 Critical assessment

Strengths. The main contribution is the correlation of heterogeneous configuration data into one graph rather than treating each finding independently. The evaluation uses 592 public Charts, the connectivity experiment directly demonstrates how graph relationships expand the attack surface, and the threat-matrix mapping gives the output an attacker-oriented reading.

Limitations. Table 4 lists the limitations we identified; the right-hand column states which of them this work addresses and where.

Table 4. Limitations of the reference paper and how this work addresses them.

#	Limitation of [1]	Addressed in this work
L1	Coefficients are chosen by hand; one setting is evaluated, with no sensitivity analysis.	Replaced by measured or learned probabilities (§6); sensitivity analysis of the remaining parameter (§7.5).
L2	Vulnerabilities are counted ($\alpha_1 = 1$ each, CVSS > 7), so 100 rarely-exploited CVEs outweigh one actively exploited CVE.	Each CVE gets an exploitation probability learned from NVD + CISA KEV (§6.3).
L3	Static analysis of Charts; no validation of the predicted edges against a live cluster or an external answer key.	Edges validated against the API server and live probes (§7.2); attack paths checked against Kubernetes Goat and Bad Pods (§7.3).
L4	Pod Security Policies are deprecated and removed (Kubernetes v1.25).	Mapped to Pod Security Standards checks (§5.2).
L5	One-off analysis; no monitoring of configuration drift.	Continuous multi-cluster monitoring with change alerts (§7.6).

4. Related work

Attack graphs. Automated attack-graph generation goes back to Sheyner et al. [8], who used model checking, and MulVAL [9], which derives attack graphs from Datalog rules over vulnerability and configuration facts. Both assume known exploit preconditions; the paper of Blaise and Rebecchi brings the idea to Kubernetes configuration.

Kubernetes auditing tools. kube-bench checks a cluster against the CIS benchmark, kube-hunter probes from an attacker’s point of view, and Checkov, kubeaudit and Polaris lint manifests [1]. They report individual findings rather than paths.

Vulnerability prioritisation. CVSS [10] measures technical severity, not the likelihood of exploitation. The Exploit Prediction Scoring System (EPSS) [11] estimates the probability of exploitation in the next 30 days, and the CISA Known Exploited Vulnerabilities (KEV) catalogue [12] lists vulnerabilities with confirmed exploitation. We use KEV as ground truth to train our model and EPSS only as an external reference.

Vulnerable-by-design clusters. Kubernetes Goat [15] is an intentionally vulnerable cluster for security training; each scenario documents an attack step by step. Bad Pods [16] is a set of eight pod manifests, each enabling one host-level setting, with documentation of whether that setting alone gives root on the node. Neither was built to evaluate attack-graph tools, which makes them a useful external answer key: their authors decided what the attacks are, not us (Section 7.3).

5. Our implementation: KubeAttackViz with static coefficients

KubeAttackViz [2] is an open-source attack-path analysis and visualisation tool for Kubernetes. It reads cluster state through kubectl (or from a JSON dump), builds a directed NetworkX graph and applies graph algorithms: BFS blast radius, Dijkstra shortest attack paths, bounded Johnson cycle detection, critical-node analysis by graph surgery, RBAC auditing, temporal comparison of snapshots, remediation advice, natural-language explanations and a D3.js interactive viewer.

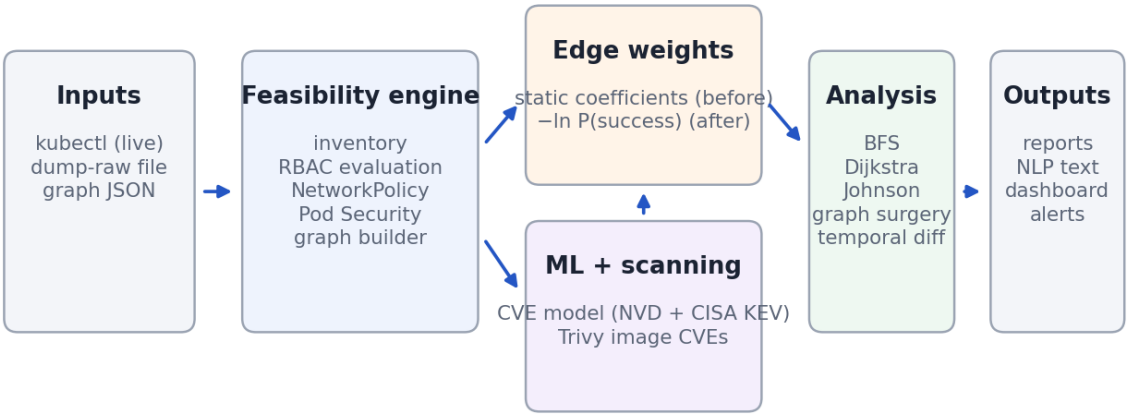


Figure 5. *KubeAttackViz architecture. The edge-weight stage is where the static coefficients (first version) are replaced by learned probabilities (this work).*

5.1 Mapping to the paper's four steps

Table 5. *Correspondence between the paper's methodology and KubeAttackViz.*

Paper step [1]	KubeAttackViz component
1. Graph extraction from a Helm Chart	Inventory read from a live cluster with kubectl (pods, ServiceAccounts, Roles, bindings, Secrets, Services, NetworkPolicies, nodes)
2. Enrichment with six features	Feasibility engine: RBAC evaluation, NetworkPolicy evaluation, pod security checks, Trivy image scanning [14]
3. Scoring (α, β, γ coefficients)	Edge weights: static coefficients (first version) → learned probabilities (this work)
4. Dijkstra on log(1/z)	Dijkstra on w = −ln P, plus BFS blast radius, cycles and critical nodes

5.2 The six features in a live cluster

The feasibility engine computes each of the paper's six features from the live API objects rather than from Chart templates:

Table 6. *The paper's six features as implemented in the feasibility engine.*

Paper feature	How KubeAttackViz computes it
1 Container vulnerabilities	Trivy scan of every running image; each CVE scored for exploitation probability (§6.3)
2 Pod accessibility	Services of type NodePort/LoadBalancer and Ingresses become edges from an internet source; RBAC users and labelled pods are also entry points
3 Security contexts	Pod Security Standards (baseline) checks: privileged, host namespaces, hostPath, capabilities, run-as-root; PSP is no longer used
4 RBAC	Full rule evaluation: scope, wildcards, resourceNames, ServiceAccount groups, token automount; e.g. read secrets, exec, create pods, bind, impersonate
5 Network policies	Ingress and egress evaluation with pod/namespace selectors (AND semantics) and ports
6 Pod affinities	Pods on the same node are linked to that node through host-access edges when they can escape the container

5.3 The first version: predefined static coefficients

Like the paper, the first version of KubeAttackViz gave every relationship type a fixed, hand-chosen cost (an “effort” value; lower means easier) and every node type a fixed impact. Path risk divided the impact of the target by the accumulated effort:

$$\mathrm{risk}_{static}(\pi) = \frac{I(t)}{1 + C(\pi)/10}, \qquad C(\pi) = \sum_{e \in \pi} c_{rel(e)} \tag{4}$$

Table 7. *Static edge coefficients of the first KubeAttackViz version (excerpt).*

Relationship	Cost	Relationship	Cost	Relationship	Cost
exposes	2.5	can_read (secret)	3.0	can_bind	5.0
runs_as	2.0	can_exec	3.5	can_proxy	5.0
bound_to	3.0	can_create_token	3.0	host_access	6.0
grants	4.0	can_impersonate	3.0	connects_to	2.0

Paths were labelled CRITICAL, HIGH, MEDIUM or LOW at risk thresholds 7, 5 and 3. This design shares the paper’s weakness L1: the numbers are plausible but not measured. A cost of 3.0 for reading a secret is the same in every cluster, whether the permission actually exists or not, and a pod with a remotely exploitable, actively exploited CVE receives the same coefficient as a pod with no known vulnerability.

6. Extension: learned, evidence-based edge weights

6.1 Design principle

We keep the paper’s probabilistic edge model and change only where the probabilities come from. Every edge e is given the probability P(e) that the corresponding attack step succeeds, and its weight is

$$w(e) = -\ln P(e), \qquad P(\pi) = \prod_{e \in \pi} P(e) = e^{-\sum_{e \in \pi} w(e)} \tag{5}$$

which is exactly the paper’s log(1/z(e)) with z = P. Dijkstra’s shortest path is therefore the most likely attack path, and the path risk becomes impact times probability of success:

$$\mathrm{risk}_{learned}(\pi) = I(t) \cdot P(\pi) \tag{6}$$

We deliberately did not train a model to imitate our own static coefficients: that would only reproduce our guesses. Instead, each edge is assigned to one of three evidence classes (Table 8).

Table 8. *Evidence classes for edge probabilities.*

Evidence class	Used for	Probability
exact-config	Steps that the configuration decides: RBAC grants, token mounts, NetworkPolicy reachability, privileged pod → node	0.995 if allowed; the edge is absent if not allowed
cve-model	Exploiting a reachable workload through a vulnerability in its image	$1 - (1 - p_u)(1 - p_x)$, p_x from the CVE model
scenario-param	Exploiting a workload with no known exploitable CVE; other host-level settings (hostPath, hostPID, hostNetwork, hostIPC)	$p_u = 0.05$ (assumed, varied in §7.5)

For exact-config edges, a value of 0.995 instead of 1 keeps every weight strictly positive. The scenario parameter p_u plays the same role as the paper’s ω: a small, non-zero chance of an unknown way in. It is the only assumed number left in the model, which is why we measure its effect in Section 7.5.

6.2 Why vulnerability counts are a poor proxy

The paper's α_1 adds 1 per high- or critical-severity CVE. Real exploitation data shows that severity is a weak signal: of 49,425 CVEs with CVSS 9–10, only 1.28% appear in the CISA KEV catalogue (Figure 6, left). Inside that same critical band, our model separates the CVEs strongly: the top fifth by model score has a 5.5% exploitation rate and the bottom fifth 0.01% (Figure 6, right). A count-based coefficient cannot make this distinction.

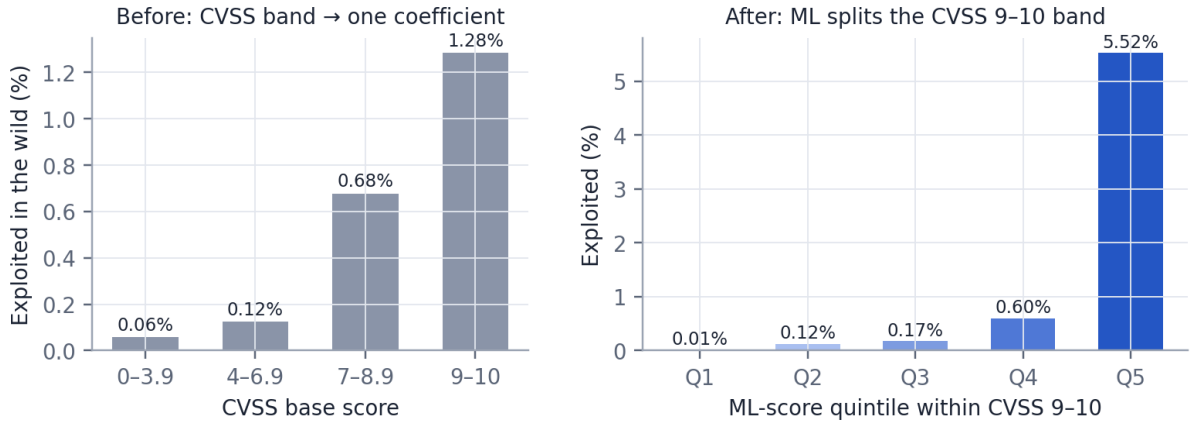


Figure 6. *Left: observed exploitation rate (CISA KEV) by CVSS band. Right: within CVSS 9–10, exploitation rate by quintile of the learned score.*

6.3 The CVE exploitation model

Data. 361,825 CVEs from the National Vulnerability Database [13] (via the FKIE mirror), labelled positive if listed in the CISA KEV catalogue (1,743 positives, catalogue version 2026.11.09). No synthetic data is used.

Features. CVSS base score and vector components (attack vector, complexity, privileges, user interaction, impacts), CWE class, publication year, reference types (vendor advisory, exploit, patch), vendor and product prevalence, and keywords from the description. Permutation importance ranks vendor advisory, base score, vendor CVE volume and CWE highest.

Model. A histogram gradient-boosting classifier (scikit-learn), which we trained ourselves and compared with a logistic regression and with CVSS score alone. To avoid leakage the evaluation is temporal: train on CVEs published up to 2023 (179,970 CVEs, 1,174 positives) and test on later CVEs (181,855 CVEs, 569 positives). A container's exploitation probability combines its CVEs:

$$p_x(\text{image}) = 1 - \prod_{i \in \text{CVEs}} (1 - \hat{p}_i) \tag{7}$$

Table 9. *Exploitation prediction on CVEs published after 2023 (base rate 0.31%).*

Model (temporal test set)	ROC-AUC	PR-AUC	Precision@100
CVSS score only (static baseline)	0.770	0.012	5%
Logistic regression	0.877	0.112	48%
Gradient boosting (deployed)	**0.887**	**0.108**	**54%**
Gradient boosting without the "in the wild" keyword	0.876	0.097	42%
FIRST EPSS (external reference, uses later data)	0.975	0.428	89%

The learned model lifts ROC-AUC from 0.770 to 0.887 and precision among the 100 highest-ranked CVEs from 5% to 54%. Removing the keyword "in the wild" from the description changes the result only

slightly (0.876), so the model is not simply reading the answer from the text. EPSS remains better; it is a production system that uses exploit and telemetry data observed after publication, so it is shown only as a reference and not as a fair competitor.

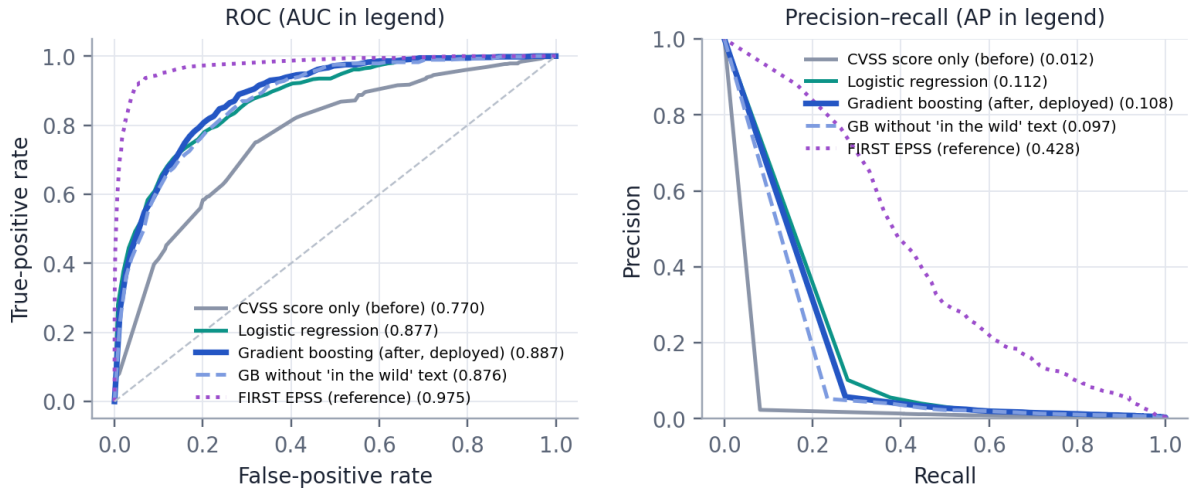


Figure 7. *ROC and precision–recall curves on the temporal test set; the grey line is the static CVSS baseline.*

7. Evaluation: static coefficients versus learned weights

7.1 Experimental setup

Table 10. *Experimental environment.*

Item	Value
Clusters	Three monitored clusters we built (kav-dev, kav-staging, kav-prod), one validation cluster (kav-lab) and one public-benchmark cluster (kav-bench), all created with kind v0.33.0 [17]
Kubernetes	Server v1.37.0, kubectl v1.37.2, CNI kindnet (kind default)
Workloads	kav-dev/staging/prod: microservices (frontend, api, payments, checkout, ledger, redis, CI runner) with deliberate misconfigurations of the kinds reported in [1]. kav-bench: unmodified Kubernetes Goat [15] and Bad Pods [16] manifests
Image scanning	Trivy v0.76.1 [14] on every running image
Graph size	15–20 nodes per monitored cluster after scoping to application namespaces; kav-bench 26 nodes (full benchmark) or 12 nodes (demonstration subset)

7.2 Validation of the edges against a live cluster

The paper’s edges are not checked against a running cluster (limitation L3). We validate ours by deploying 26 scenarios on the lab cluster and comparing every predicted edge with the cluster’s own answer: the API server’s SubjectAccessReview for RBAC permissions, the presence of a mounted token, and real HTTP probes between pods for network reachability.

Table 11. Predicted edges compared with ground truth from a live cluster.

Check	Checks	Precision	Recall	Accuracy
RBAC permissions (SubjectAccessReview)	21,019	1.000	1.000	1.000
ServiceAccount token availability	190	1.000	1.000	1.000
Network reachability, ordinary pods	1,050	1.000	1.000	1.000
Network reachability, hostNetwork pods	166	0.873	0.605	0.669
Overall	**22,425**	**0.996**	**0.982**	**0.9975**

All RBAC, token and ordinary network edges are exact. The 55 disagreements all involve pods using the host network, whose traffic Kubernetes NetworkPolicy semantics do not cover consistently across network plugins; we report this as a known limitation rather than tuning it away.

7.3 Public benchmark: Kubernetes Goat and Bad Pods

Section 7.2 checks single edges. To check whole attack paths we need an answer key, and on clusters we built ourselves we would also be writing that answer key. We therefore use two public projects whose authors already documented which attacks work: Kubernetes Goat [15] and Bad Pods [16]. Their manifests are deployed **unmodified** on the kav-bench cluster, with two mechanical exceptions: the one Goat component that upstream installs with Helm is rendered from its chart's default values, and the Bad Pods, which have no namespace, are applied in a namespace of their own.

Answer key. From Kubernetes Goat we take the seven scenarios whose attack follows from cluster configuration (2, 3, 4, 8, 11, 12, 16). The others concern image contents, CIS or runtime tools, denial of service or a deprecated Helm v2 component and are out of scope for a configuration graph. From Bad Pods we take all eight pods; the documentation of each states whether that pod's specification alone gives root on its node (four yes, four no). Every entry of the answer key links to the page where it is documented (Table 15).

Procedure. The tool scans the live cluster once and builds the graph under both weightings. For each documented attack it reports whether the path is in the graph and its rank among all attack paths of the cluster. Following Kubernetes Goat's premise that any application may be compromised, entry points are the internet, users and every pod, which gives 80 entry-to-target paths. Risks are compared at the two decimals shown, so equal risks share a rank. For Bad Pods we ask whether each weighting claims an escape on the direct pod → node step: the static weights have no probability, so any modelled step counts as a claim, while the learned weights claim an escape when $P \geq 0.5$.

Table 12. Kubernetes Goat: documented attacks found and their rank among 80 paths (lower is more dangerous). "reachable": the target is a workload, not a secret or node, so the path is checked but not ranked.

Goat scenario	Documented attack	Found	Old rank	ML rank	ML P(path)
#2 DIND (docker-in-docker) exploitation	escape out of the health-check container to the host system	yes	#20	#1	99.5%
#3 SSRF in the Kubernetes world	from the internal-proxy app, query the internal-only metadata-db service	yes	reachable	reachable	99.1%
#4 Container escape to the host system	escape out of the system-monitor container to the host	yes	#20	#1	99.5%
#8 NodePort exposed services	reach an internal service from outside through NodePort 30003	yes	reachable	reachable	75.5%
#11 Kubernetes namespaces bypass	from a pod in the default namespace, reach cache-store in secure-middleware	yes	reachable	reachable	75.5%
#12 Gaining environment information	read the vault key from system-monitor's environment (Secret goatvault)	yes	#1	#1	99.5%
#16 RBAC least privileges misconfiguration	use the hunger-check ServiceAccount token to read Secret vaultapikey	yes	#29	#7	98.5%

Table 13. Bad Pods: can the pod's own specification reach its node? Green: agrees with the documentation; red: disagrees.

Bad Pod	Documented outcome	Expected	Old weights say	ML weights say	ML P
#1 Everything allowed	chroot into the mounted host filesystem gives root on the node	escape	escape	escape	99.5%
#2 Privileged and hostPID	nsenter into PID 1 gives a root shell on the node	escape	escape	escape	99.5%
#3 Privileged only	mount the host disk or use a cgroup escape for root on the node	escape	escape	escape	99.5%
#4 hostPath only	mounting / gives read/write access to the host filesystem	escape	escape	no escape	5.0%
#5 hostPID only	no clear path to root on the node (only process/env snooping)	no escape	escape	no escape	5.0%
#6 hostNetwork only	no privileged code execution on the host directly (sniffing, localhost services)	no escape	escape	no escape	5.0%
#7 hostIPC only	you most likely can't do much (shared memory only)	no escape	escape	no escape	5.0%
#8 Nothing allowed	host namespaces and capabilities all blocked	no escape	no escape	no escape	0.0%

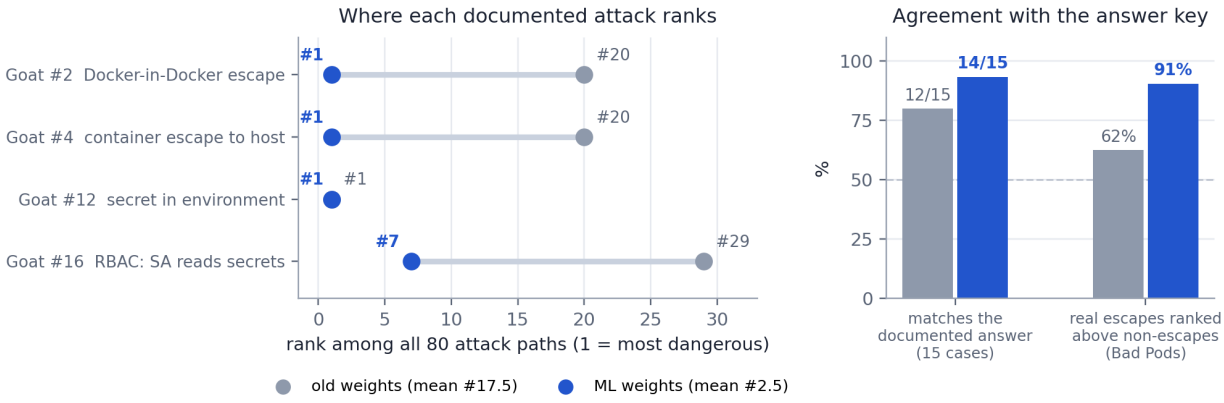


Figure 8. *Left: rank of each ranked Kubernetes Goat attack under old and ML weights. Right: agreement with the 15 documented answers, and the share of (real escape, non-escape) Bad Pod pairs that each weighting orders correctly; the dashed line is chance (50%).*

Table 14. *Benchmark summary (full benchmark, Trivy image scanning on).*

Measure	Old weights (paper-style)	ML weights
Kubernetes Goat attacks found (recall)	7/7	**7/7**
Mean rank of documented Goat attacks (of 80)	17.5	**2.5**
Bad Pods verdicts that match the documentation	5/8	**7/8**
Real escapes ranked above non-escapes	62%	**91%**
All documented answers matched	**12/15**	**14/15**

Reading the results. Both weightings find every documented Kubernetes Goat attack. This is expected: weights change the order of paths, not which edges exist, so recall is a property of the feasibility engine. The difference is the ranking. The learned weights put three of the four ranked attacks first and the fourth (reading secrets through the hunger-check ServiceAccount) seventh; the static weights place them 1st, 20th, 20th and 29th. The seventh place is itself informative: the Goat images contain CVEs listed in the CISA KEV catalogue, so the learned model also ranks many “compromise system-monitor through a known-exploited CVE” paths near the top with almost the same probability.

On Bad Pods the static weights cannot tell host settings apart: every pod with any host-level setting gets the same pod → node edge, so hostPID, hostNetwork and hostIPC are all reported as escapes, against the documentation. The learned weights treat privileged pods as configuration-decided (P = 0.995) and the other settings as uncertain (P = 0.05), which matches Bad Pods for seven of eight pods. The one disagreement is **hostPath**: Bad Pods documents that mounting the host’s root directory gives read/write access to the node, but our model treats every hostPath mount as uncertain, because many hostPath mounts are read-only or harmless directories. Treating writable mounts of sensitive host paths as configuration-decided is the obvious fix; we report the miss rather than adjusting the model after seeing the answer key.

Demonstration subset. For the live demonstration (Section 8) kav-bench runs a 12-node subset: three Goat applications and two Bad Pods (privileged, hostPID). On it the learned weights match 6 of 6 documented answers and the static weights 5 of 6, but the subset contains no hostPath pod, so the numbers reported in this section are those of the full benchmark.

Table 15. Sources of the answer key; each page states the attack used as the expected answer.

Entry	Documented at
Goat #2	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-2/scenario-2.md
Goat #3	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-3/scenario-3.md
Goat #4	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-4/scenario-4.md
Goat #8	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-8/scenario-8.md
Goat #11	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-11/scenario-11.md
Goat #12	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-12/scenario-12.md
Goat #16	https://github.com/madhuakula/kubernetes-goat/blob/master/guide/docs/scenarios/scenario-16/scenario-16.md
Bad Pod #1	https://github.com/BishopFox/badPods/tree/main/manifests/everything-allowed
Bad Pod #2	https://github.com/BishopFox/badPods/tree/main/manifests/priv-and-hostpid
Bad Pod #3	https://github.com/BishopFox/badPods/tree/main/manifests/priv
Bad Pod #4	https://github.com/BishopFox/badPods/tree/main/manifests/hostpath
Bad Pod #5	https://github.com/BishopFox/badPods/tree/main/manifests/hostpid
Bad Pod #6	https://github.com/BishopFox/badPods/tree/main/manifests/hostnetwork
Bad Pod #7	https://github.com/BishopFox/badPods/tree/main/manifests/hostipc
Bad Pod #8	https://github.com/BishopFox/badPods/tree/main/manifests/nothing-allowed

7.4 Before and after on three live clusters

Both weightings were run on the same graphs of our three monitored clusters. Table 16 compares the paths found and their ordering; Figure 9 shows the severity labels assigned under each model.

Table 16. Static versus learned weighting on the live clusters (ρ = Spearman rank correlation of path risks, measured on 12 November 2026 with Trivy image scanning).

Cluster	Nodes / edges	Paths	Severity, static (before)	Severity, learned (after)	Rank corr. ρ
kav-dev	20 / 51	16	13 high, 3 medium	16 critical	0.71
kav-staging	19 / 38	11	8 high, 3 medium	8 critical, 3 medium	0.84
kav-prod	19 / 28	19	5 high, 14 medium	4 critical, 8 medium, 7 low	0.57

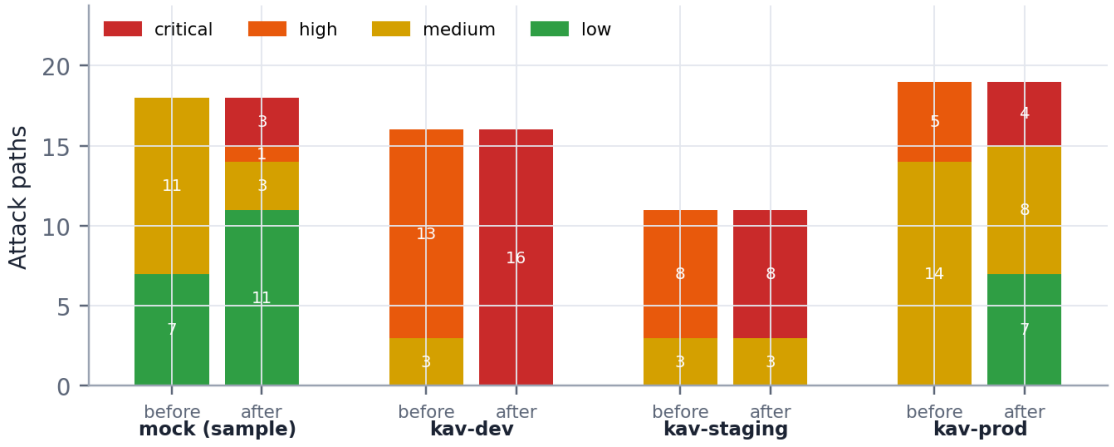


Figure 9. Severity of the attack paths before (static coefficients) and after (learned weights). “mock” is the sample cluster shipped with the tool.

The orderings differ where it matters; the rank correlation between the two models is lowest in kav-prod (0.57). Table 17 lists the two top-ranked paths of each cluster. With static coefficients, the short path ci-runner → registry-token is first everywhere because it has only one edge. With learned weights, the first path in kav-dev is dev-bob → platform-admins → cluster-admin: every step is a verified RBAC grant, so the path succeeds with probability 0.99 and leads to full cluster control. In kav-prod, the learned model promotes the break-glass role of sre-carol, which reaches ledger and payment credentials with probability 0.985, while the static model ranks the network chain ci-runner → frontend → api → session-key second even though its pod-to-pod exploitation steps have a low measured probability.

Table 17. *Two highest-ranked paths per cluster under each weighting.*

Cluster	Top paths — static (risk)	Top paths — learned (P of success)
kav-dev	ci-runner → registry-token (6.67) dev-bob → platform-admins → cluster-admin (5.88)	dev-bob → platform-admins → cluster-admin (0.990) ci-runner → ci-runner → platform-admins → cluster-admin (0.985)
kav-staging	ci-runner → registry-token (6.67) ci-runner → api → session-key (5.81)	ci-runner → registry-token (0.995) qa-alice → qa-alice-edit → session-key (0.990)
kav-prod	ci-runner → registry-token (6.67) ci-runner → frontend → api → session-key (5.14)	ci-runner → registry-token (0.995) sre-carol → sre-carol-breakglass → ledger → ledger-creds (0.985)

Severity becomes interpretable. Under static coefficients, risk is a ratio of hand-chosen numbers and the HIGH/MEDIUM labels have no external meaning. Under learned weights, a CRITICAL path is one that reaches a high-impact target with a high, evidence-backed probability of success. In kav-prod, which has network policies and hardened pods, the learned model labels seven paths LOW because they depend on exploiting workloads whose images have a low measured exploitation probability; the static model could not distinguish these from configuration-guaranteed paths.

The same graph, two orderings. Figure 10 draws the 41-node sample cluster shipped with the tool under both weightings with one layout, highlighting each weighting’s four highest-ranked paths. The static weights favour a chain through sa-worker → pod-exec → worker-node-1; under learned weights that chain drops out of the top four (the former first path falls to 15th, P = 0.7%) and the credential chain from cicd-bot to production-db, previously fourth, becomes first with P = 52.4%.

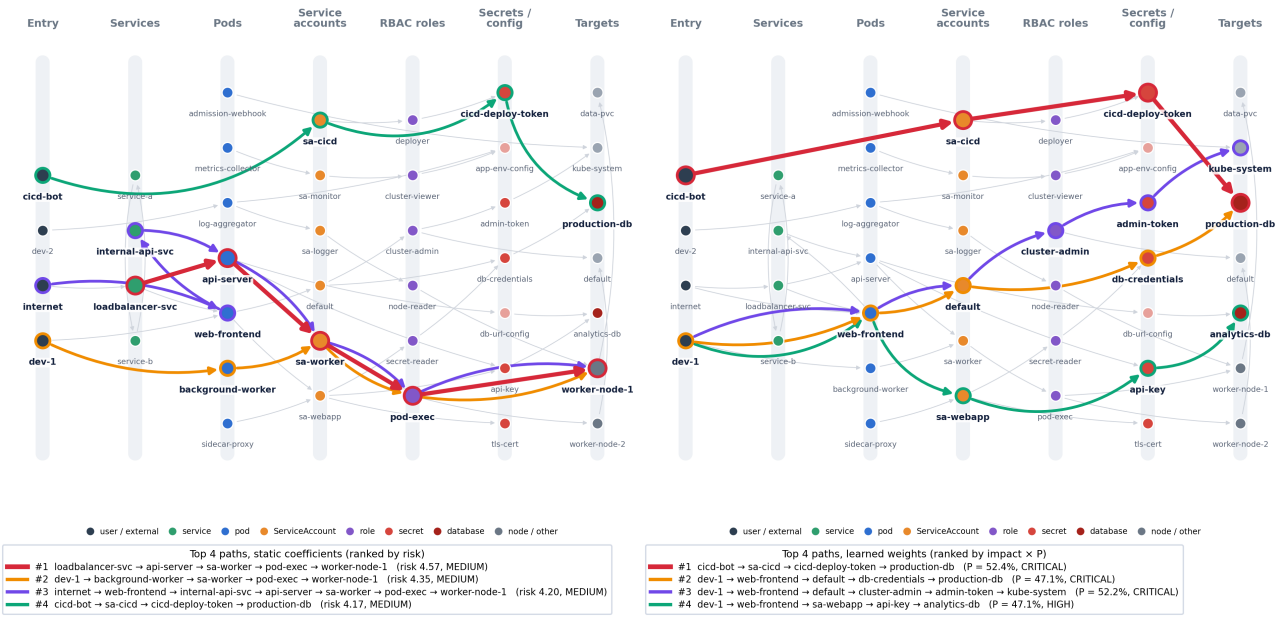


Figure 10. *The sample cluster under (a) static coefficients and (b) learned weights, drawn with one layout; each panel highlights its four highest-ranked paths (#1 red, #2 orange, #3 purple, #4 green).*

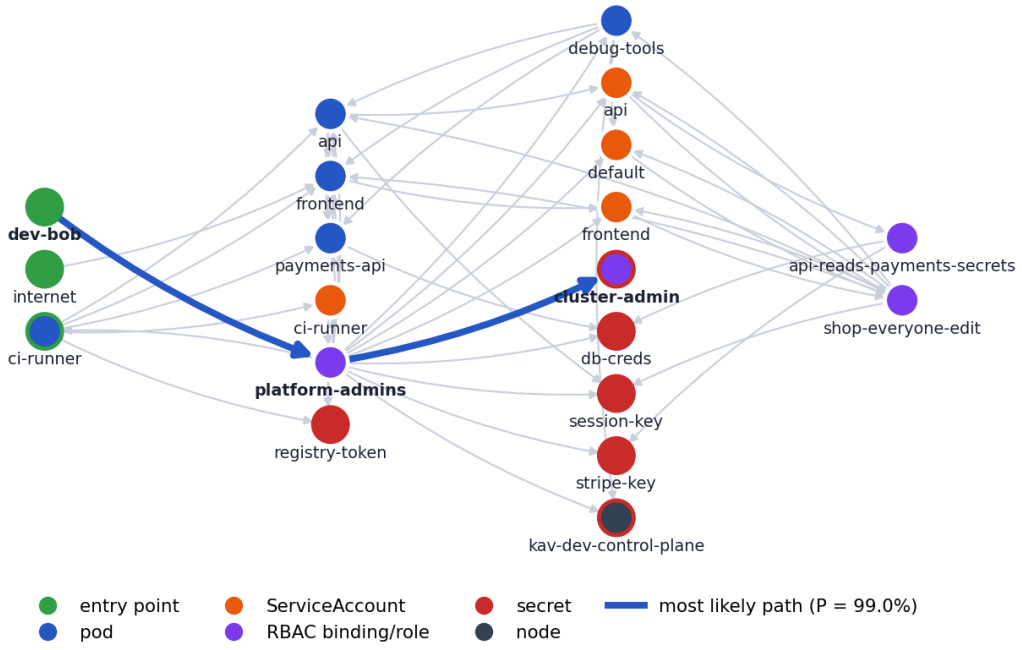


Figure 11. *Live kav-dev attack graph (20 nodes). The highlighted path is the most likely one under learned weights (P = 0.99).*

7.5 Sensitivity analysis

The paper evaluates one coefficient setting (limitation L1). Our model has one assumed parameter, p_u. We varied it from 0.01 to 0.5 and measured the Kendall rank correlation τ of the path ordering against the default ($p_u = 0.05$).

Table 18. Kendall τ of path rankings against the default $p_u = 0.05$.

p_u	kav-dev τ	kav-staging τ	kav-prod τ	Top path unchanged
0.01	1.000	1.000	1.000	yes
0.02	1.000	1.000	1.000	yes
0.05 (default)	1.000	1.000	1.000	yes
0.10	1.000	1.000	1.000	yes
0.20	1.000	1.000	1.000	yes
0.30	1.000	1.000	1.000	yes
0.50	0.962	0.990	1.000	yes

The ranking is identical (τ = 1) for every p_u up to 0.3, and the most dangerous path never changes. Only at $p_u = 0.5$, which assumes that half of all unpatched-looking services are exploitable, do a few lower-ranked paths swap. The conclusions therefore rest on measured evidence rather than on the assumed parameter.

7.6 Real-time detection of a misconfiguration

The paper analyses Charts once, before deployment (limitation L5). KubeAttackViz rescans each monitored cluster every 15 seconds, recomputes the graph with both weightings and compares it with the previous snapshot. To test this end to end we introduced a realistic misconfiguration into the hardened kav-prod cluster while it was being monitored: a ClusterRoleBinding giving the ServiceAccount of the shop/frontend pod the cluster-admin role. We then removed it again.

Table 19. Runtime drift experiment on kav-prod.

Step	Observation
Binding created	Detected after 16 s (one 15 s scan interval). The graph grew from 19 to 21 attack paths and the monitor raised CRITICAL alerts such as “New attack path: ci-runner → … → cluster-admin”.
New path, static weights	ci-runner → ci-runner-deploys-shop → frontend → kav-drift-frontend-admin → cluster-admin ranked **#17, MEDIUM** (risk 3.85).
New path, learned weights	The same path ranked **#1, CRITICAL** (P = 0.975): every step is a verified RBAC grant or token mount.
Binding removed	Detected after 12 s; the graph returned to its previous 19 paths.

Granting cluster-admin to a workload identity is among the most serious Kubernetes misconfigurations, yet the static weights rank the resulting path in the lower half of the list because it is long. The learned weights rank it first because each of its steps is decided by the configuration. This addresses the paper’s CI/CD future-work direction from the runtime side.

8. Demonstration

All experiments in Section 7 can be repeated live from one command, ./demo/present.sh. It shows one menu: the clusters with their posture at the top (switched with s and a–d), and five actions below. All of them only read from the clusters.

Table 20. The demonstration menu.

Action	What it shows
1 Show pods	The workloads, their status, ServiceAccount and image
2 Show RBAC & NetworkPolicies	Who can do what, and whether pods are isolated on the network
3 Live monitor + dashboard	Continuous scanning with alerts, and the interactive graph in the browser
4 Old vs ML weights	Four dashboard views — the sample cluster and the selected live cluster, each under static and learned weights — with one layout per cluster, the four highest-ranked paths in colour and each path tagged with its rank under the other weighting
5 Benchmark	The comparison of Section 7.3 against Kubernetes Goat and Bad Pods, with the link to every documented answer

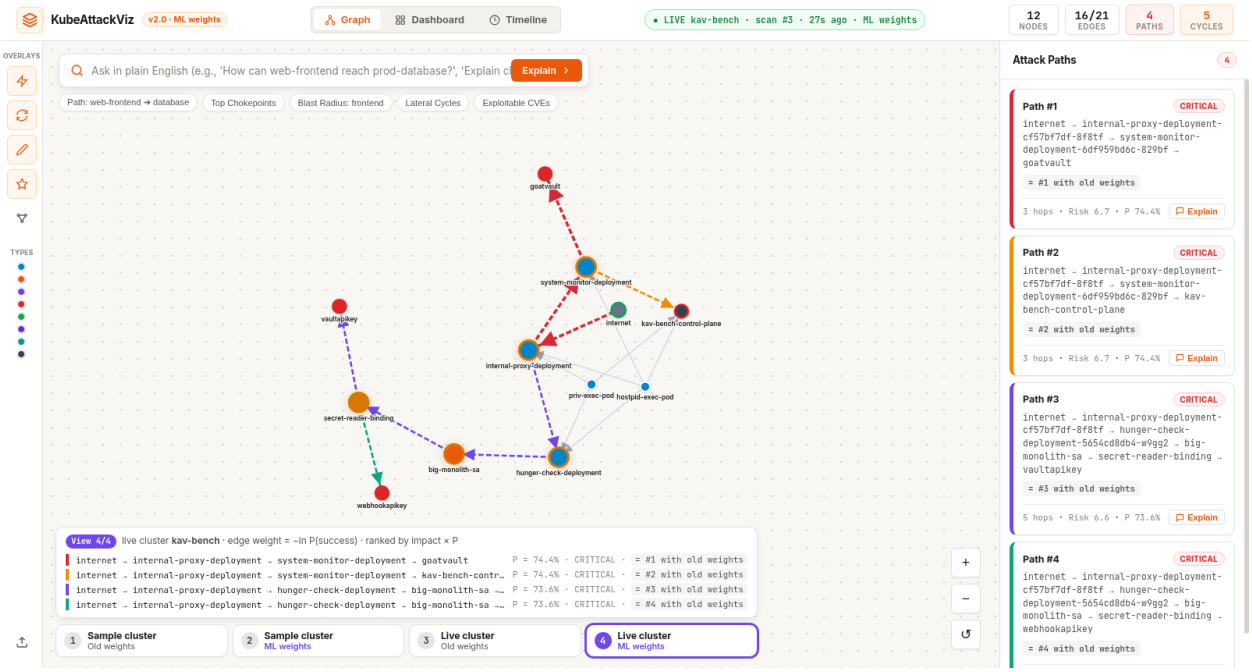


Figure 12. *Dashboard view 4 (live kav-bench cluster, learned weights). Each path card and the panel at the bottom state the rank the path had under static weights. Only edges that lie on attack paths, plus the closest others, are drawn for readability; the analysis uses every edge, and a switch on the left shows them all.*

9. Discussion

9.1 Relationship to the paper

The research contribution, the graph formulation and the log-probability path search come from Blaise and Rebecchi [1]. Our work keeps that formulation and changes the source of the numbers. Configuration-decidable steps are proven from the live cluster rather than scored, and the vulnerability step uses a probability learned from public exploitation data rather than a count of severe CVEs. Table 21 summarises the comparison.

Table 21. Comparison of the reference paper with both versions of our implementation.

Aspect	Stay at the Helm [1]	KubeAttackViz, static (v1)	KubeAttackViz, learned (this work)
Input	Helm Charts	Live cluster / JSON	Live clusters, monitored continuously
Edge weights	Predefined γ coefficients	Predefined costs per relation	−ln P(e), proven or learned
Vulnerabilities	Count of CVSS > 7 CVEs (α_1)	Not used	Exploitation probability from NVD + CISA KEV
Pod security	Pod Security Policies	Pod flags	Pod Security Standards (baseline)
Path search	Dijkstra on log(1/z)	Dijkstra on effort	Dijkstra on −ln P
Edge validation	Statistics over 592 Charts	Sample data	22,425 checks on a live cluster
External answer key	None	12/15 documented answers	14/15 documented answers
Sensitivity analysis	None	None	p_u from 0.01 to 0.5
Scale of evaluation	1,318 containers	Demo clusters	Three clusters of 15–20 nodes + public benchmark

9.2 Threats to validity

- **Scale.** The paper evaluates 592 Charts; our live comparison uses three small clusters that we built and a public benchmark of 15 documented cases. The results show how the weighting changes rankings and that it agrees better with an external answer key, not how frequent each misconfiguration is in practice.
- **Answer key.** Kubernetes Goat and Bad Pods document the attacks they were designed for. Paths our tool finds beyond those are not necessarily wrong, so we report recall and rank rather than precision. Ranking the documented attacks highly is a reasonable but not a neutral criterion: it assumes the documented attacks are among the most dangerous in their cluster.
- **hostPath.** The learned model rates every hostPath mount as uncertain (P = 0.05), which disagrees with Bad Pods for a writable mount of the host root (Section 7.3).
- **Label noise.** CISA KEV lists confirmed exploitation only; many exploited CVEs are never listed, so absolute probabilities are conservative. Ranking metrics are less affected than calibration.
- **Independence.** Multiplying edge probabilities assumes independent steps, as in the paper. Correlated steps (for example, the same stolen token used twice) are approximated.
- **Host networking.** Reachability for hostNetwork pods is not modelled exactly (Section 7.2).

10. Conclusion and future work

Blaise and Rebecchi show that Kubernetes configuration security is better understood as a connected graph of resources, permissions and movement opportunities than as a list of isolated checks, and that paths can be found with Dijkstra after a log transformation of edge probabilities [1]. Their probabilities, like the weights in our first implementation, are predefined coefficients.

This work keeps the paper’s formulation and replaces those coefficients with evidence. Steps decided by the configuration are verified against the live cluster (all 21,019 RBAC checks correct), and the vulnerability step uses a model trained on 361,825 real CVEs that improves on CVSS alone from 0.770 to 0.887 ROC-AUC and from 5% to 54% precision in the top 100. On attack paths documented by Kubernetes Goat and Bad Pods, which we did not write, the learned weights match 14 of 15 documented answers against 12 and rank the documented Goat attacks at a mean position of 2.5 instead of 17.5. On

three live clusters they reorder the attack paths (rank correlation 0.57–0.84 with the static model), promote configuration-guaranteed paths to cluster-admin and production secrets, and demote paths that rely on unlikely exploits; a cluster-admin misconfiguration introduced at runtime is detected within one scan interval and ranked first only by the learned weights. The only remaining assumed parameter does not change the rankings for any value up to 0.3.

Future work. (i) Treat writable hostPath mounts of sensitive host directories as configuration-decided, the one disagreement with the benchmark; (ii) parse Helm Charts directly (through helm template) so that the paper's 592-Chart dataset can be re-evaluated with learned weights; (iii) model the paper's per-tactic sub-scores with tactic-specific probabilities; (iv) handle hostNetwork reachability per network plugin; (v) integrate the analysis as an admission or CI/CD gate, as proposed in [1].

References

[1] A. Blaise and F. Rebecchi, "Stay at the Helm: secure Kubernetes deployments via graph generation and attack reconstruction," in *Proc. 2022 IEEE 15th Int. Conf. on Cloud Computing (CLOUD)*, Barcelona, Spain, 2022, pp. 59–69, doi: 10.1109/CLOUD55607.2022.00022.

[2] BugBuster18, "Kuber_h2f: KubeAttackViz — Kubernetes attack path visualizer," GitHub repository, 2026. https://github.com/BugBuster18/Kuber_h2f

[3] Microsoft, "Threat matrix for Kubernetes," Microsoft Security Blog, Apr. 2020. https://www.microsoft.com/security/blog/2020/04/02/attack-matrix-kubernetes/

[4] The MITRE Corporation, "MITRE ATT&CK." https://attack.mitre.org/

[5] Center for Internet Security, "CIS Kubernetes Benchmark." https://www.cisecurity.org/benchmark/kubernetes

[6] The Kubernetes Authors, "Using RBAC authorization," Kubernetes documentation. https://kubernetes.io/docs/reference/access-authn-authz/rbac/

[7] The Kubernetes Authors, "Pod Security Standards," Kubernetes documentation. https://kubernetes.io/docs/concepts/security/pod-security-standards/

[8] O. Sheyner, J. Haines, S. Jha, R. Lippmann and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. on Security and Privacy*, 2002, pp. 273–284.

[9] X. Ou, S. Govindavajhala and A. W. Appel, "MulVAL: a logic-based network security analyzer," in *Proc. 14th USENIX Security Symp.*, 2005.

[10] FIRST, "Common Vulnerability Scoring System v3.1: specification document." https://www.first.org/cvss/

[11] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid and M. Roytman, "Exploit Prediction Scoring System (EPSS)," *Digital Threats: Research and Practice*, vol. 2, no. 3, 2021.

[12] Cybersecurity and Infrastructure Security Agency, "Known Exploited Vulnerabilities catalog." https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[13] NIST, "National Vulnerability Database." https://nvd.nist.gov/

[14] Aqua Security, "Trivy: vulnerability scanner for containers." https://github.com/aquasecurity/trivy

[15] M. Akula, "Kubernetes Goat: interactive Kubernetes security learning playground," GitHub repository (MIT licence), commit 723a0db, 2026. https://github.com/madhuakula/kubernetes-goat

[16] Bishop Fox, "Bad Pods: Kubernetes pod privilege escalation," GitHub repository (MIT licence), commit 107e5d8, 2025. https://github.com/BishopFox/badPods

[17] The Kubernetes Authors, "kind: Kubernetes IN Docker." https://kind.sigs.k8s.io/